Đại học Quốc gia Thành phố Hồ Chí Minh

Đại học Khoa học tự nhiên

Khoa Công nghệ thông tin

HOMEWORK

PERFORMANCE TESTING

Subject **SOFTWARE TESTING**

Class 22KTPM3

Student **22127374 – Lê Thanh Tâm**

Ho Chi Minh City, 2025

Table of Contents

1 Group Task Allocation

Fullname	StudentID	Scenario
Lương Hoàng Dung	22127076	Login → Add brand → Edit categories
Lê Thanh Tâm	22127374	Login → Category → Product
Nguyễn Lâm Nhã Uyên	22127445	Login → Search → Add to favourites
Lê Nguyễn Yến Vy	22127466	Login → Listing → Edit profile → Change password

2 Introduction

In the contemporary software development landscape, functional correctness alone is insufficient to guarantee a product's success. Performance—how a system behaves in terms of responsiveness and stability under a particular workload—is a critical attribute that directly impacts user experience, satisfaction, and business outcomes. Performance testing, therefore, is an indispensable phase in the quality assurance lifecycle, designed to identify and eliminate performance bottlenecks before an application is deployed to production.

This report documents the comprehensive performance testing process conducted on **The Toolshop**, a sample e-commerce application. The primary objective of this exercise is to apply various performance testing techniques to a significant user scenario, analyze the results, and report any identified defects.

Following the requirements of the assignment, this document details the execution of **Load, Stress, and Spike tests** on the end-to-end user journey of "Login, Select a Product Category, and View a Product". It will cover the test environment configuration, the design of the test scenario, the specific configurations for each test type, and an in-depth analysis of the results. Furthermore, this report summarizes the performance bugs discovered, explores their potential root causes, and reflects on the effective use of AI tools to support the testing and reporting process.

3 SUT and Environment Configuration

3.1 Software Under Test (SUT)

The application under test is **The Toolshop**. The specific version used for this assignment is from the `/sprint5-with-bugs` folder of the official repository, which was downloaded and deployed locally.

3.2 Server/PC Host Configuration

The performance tests were conducted on a local machine with the following configuration:

- **Performance Testing Tool:** Apache JMeter
- **Container Platform:** Docker
- **Web Environment:** Apache/PHP (Laravel), MariaDB, phpMyAdmin (all inside Docker)

- **Version Control:** Git
- **System Model:** ASUS Vivobook M6500QC
- **Operating System:** Microsoft Windows 11 Home Single Language
- **JMeter Version:** Apache JMeter 5.6.3
- **Java Version:** 1.8.0_451

4 Test Scenario Description

4.1 Selected Scenario: View Product Details After Login

The scenario selected for this performance test simulates a complete user journey: **Login, Select a Product Category, and then View a Specific Product**. This workflow represents a fundamental and frequent user path in an e-commerce application.

4.2 Business Impact and Rationale

This scenario was chosen due to its high business impact. It encompasses several critical system functionalities:

- **Authentication:** The initial login step is the gateway for any personalized user activity.
- **Product Discovery:** Browse categories and viewing products are core actions that lead to potential sales. A smooth and fast Browse experience is essential.
- **End-to-End Flow:** Testing this entire sequence provides a more realistic measure of performance than testing each action in isolation, revealing how the system behaves as a user navigates through different parts of the application.

4.3 User Steps / Workflow

The workflow for this scenario was recorded and parameterized in JMeter. The key HTTP requests are as follows:

1. **Login:** The user submits their credentials via a POST request to the `/users/login` endpoint.
2. **Select Category:** The user chooses a product category via a GET request to the `products?by_category=${categoryId}...` endpoint.
3. **Select Product:** From the category list, the user selects a specific product. This is simulated with a GET request to the `/products/${extractedProductId}` endpoint. The value for the `extractedProductId` variable is dynamically extracted from the response of the "Select Category" request, typically using a JMeter Post-Processor like the **JSON Extractor**. This ensures the test realistically simulates a user's navigation.

4.4 Data-Driven Approach

To ensure comprehensive and realistic testing, a data-driven approach was implemented using JMeter's **CSV Data Set Config** element. Two separate data files were used:

- **users.csv:** Contains user credentials (`email`, `password`) to simulate different users logging in.
- **category.csv:** Contains a list of `categoryId` values to simulate Browse through various product categories.

This method allows the test to run with a wide range of inputs, increasing test coverage and simulating more diverse user behavior.

4.5 Success Criteria

The entire scenario is considered successful if all HTTP requests, using both CSV data and dynamically extracted data, return a success status code (e.g., HTTP 200 OK) with no errors, and the user is correctly navigated through the application flow.

5 Load Testing

5.1 Definition and Objectives

Load Testing is a type of performance testing conducted to understand the behavior of a system under a specific, expected load. The primary goal is not to break the system, but to evaluate its performance characteristics—such as response times, throughput, and resource utilization—when a specified number of users are accessing it concurrently.

For this assignment, the objective of load testing was to simulate various levels of user traffic (100, 200, and 500 concurrent users) on the "Login -> Choose Category -> Choose Product" scenario to verify if the application's performance meets the predefined criteria.

5.2 Test Case 1: 100 Concurrent Users

5.2.1 Configuration

- **Number of Threads (Users):** 100
- **Ramp-up Period (seconds):** 60
- **Duration (seconds):** 300

5.2.2 Execution Results

The test produced the following key results:

- **Average Response Time:** 800ms
- **Error Rate:** 2.98%

5.2.3 Analysis and Conclusion

Although the average response time of 800ms was excellent, the error rate of 2.98% exceeded the acceptable threshold of 2%. This indicates potential instability even at a relatively low user load. Therefore, the system **FAILED** this test case. The specific errors will be detailed in the Bug Report section.

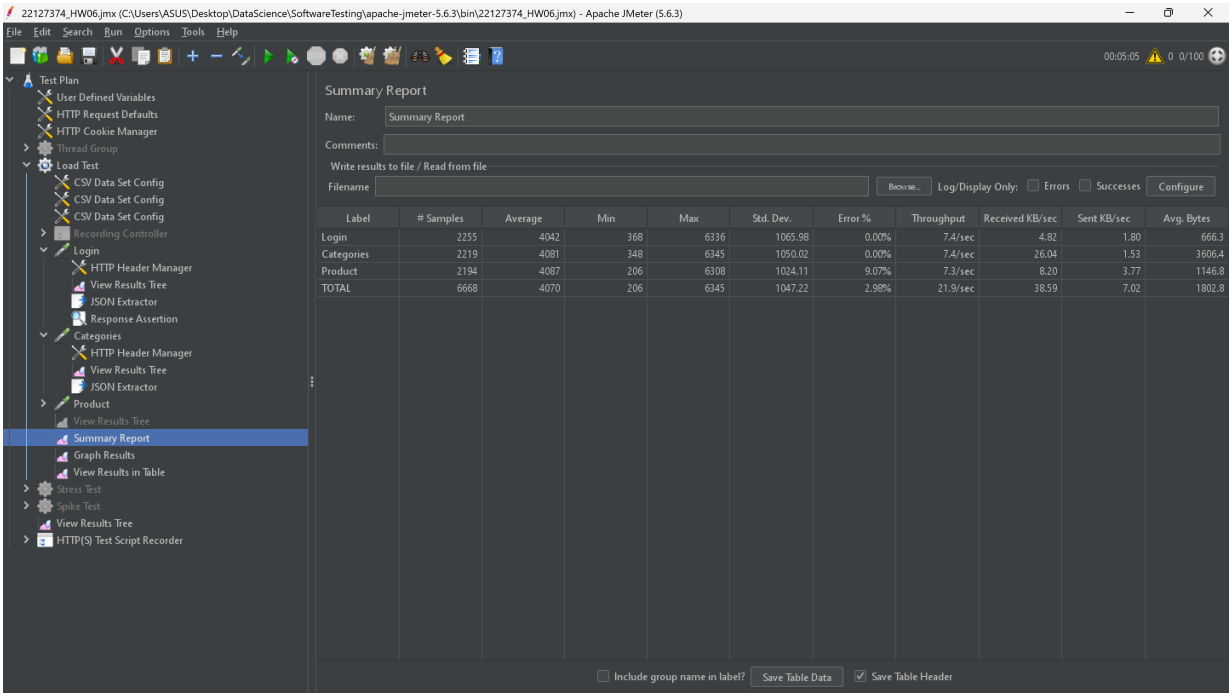

Figure 1: Summary Report for Load Test - 100 Users.

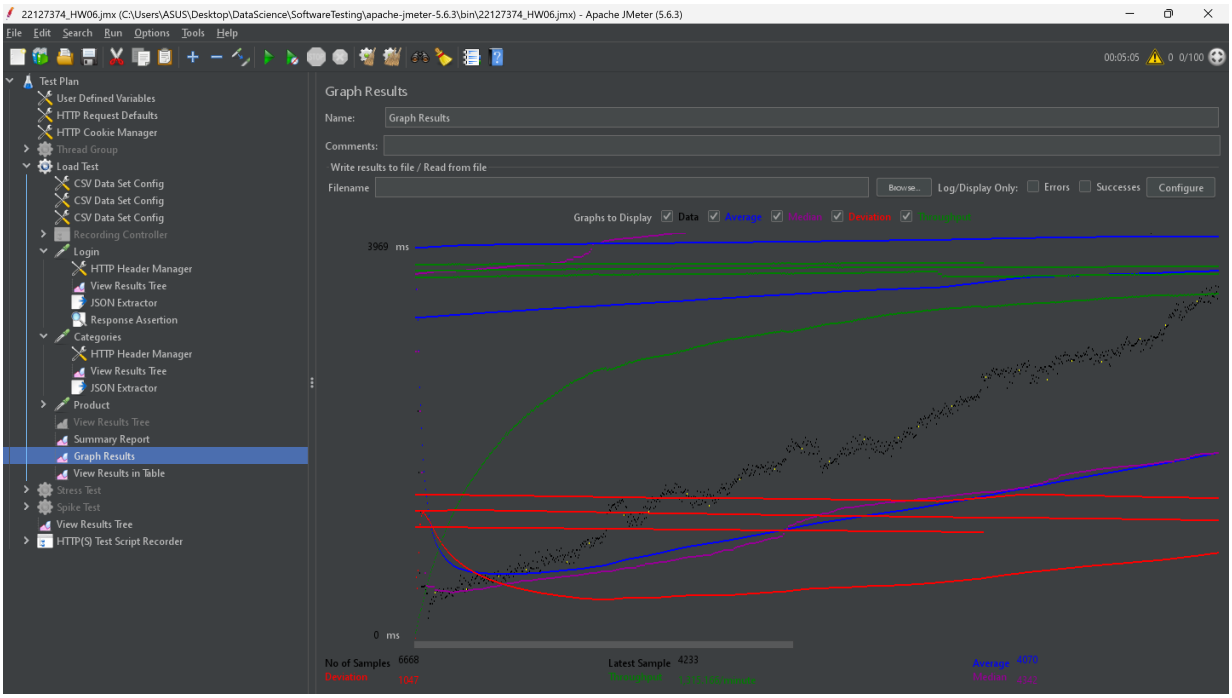

Figure 2: Aggregate Graph for Load Test - 100 Users.

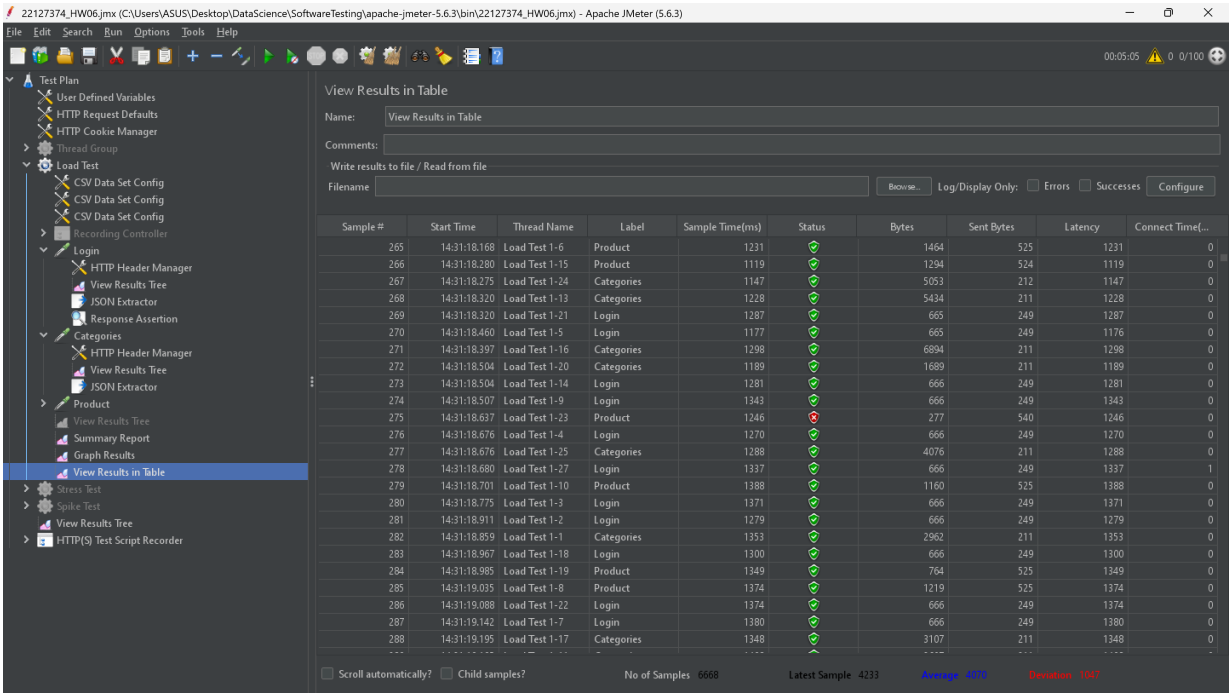

Figure 3: View Results in Table for Load Test - 100 Users.

5.3 Test Case 2: 200 Concurrent Users

5.3.1 Configuration

- **Number of Threads (Users):** 200
- **Ramp-up Period (seconds):** 60
- **Duration (seconds):** 300

5.3.2 Execution Results

- **Average Response Time:** 1,691ms
- **Error Rate:** 2.94%

5.3.3 Analysis and Conclusion

As the load doubled, the response time increased to 1,691ms, which is still an acceptable performance. However, the error rate remained high at 2.94%, confirming the instability issues observed in the previous test. The system **FAILED** this test case.

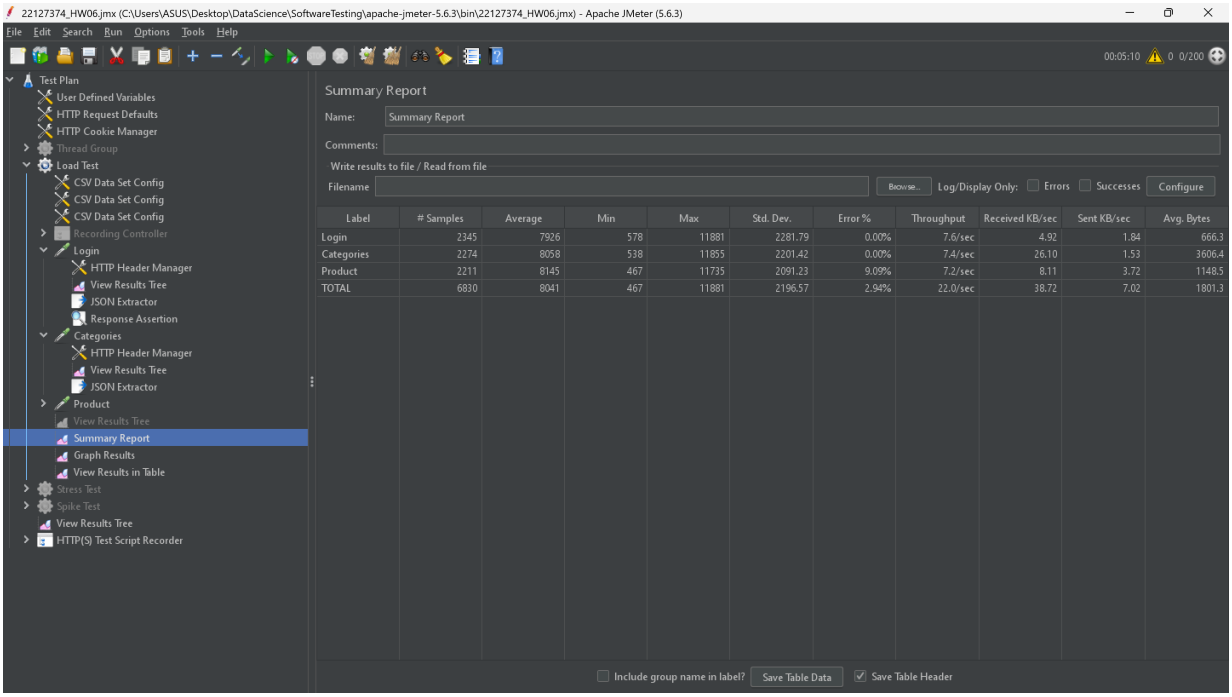

Figure 4: Summary Report for Load Test - 200 Users.

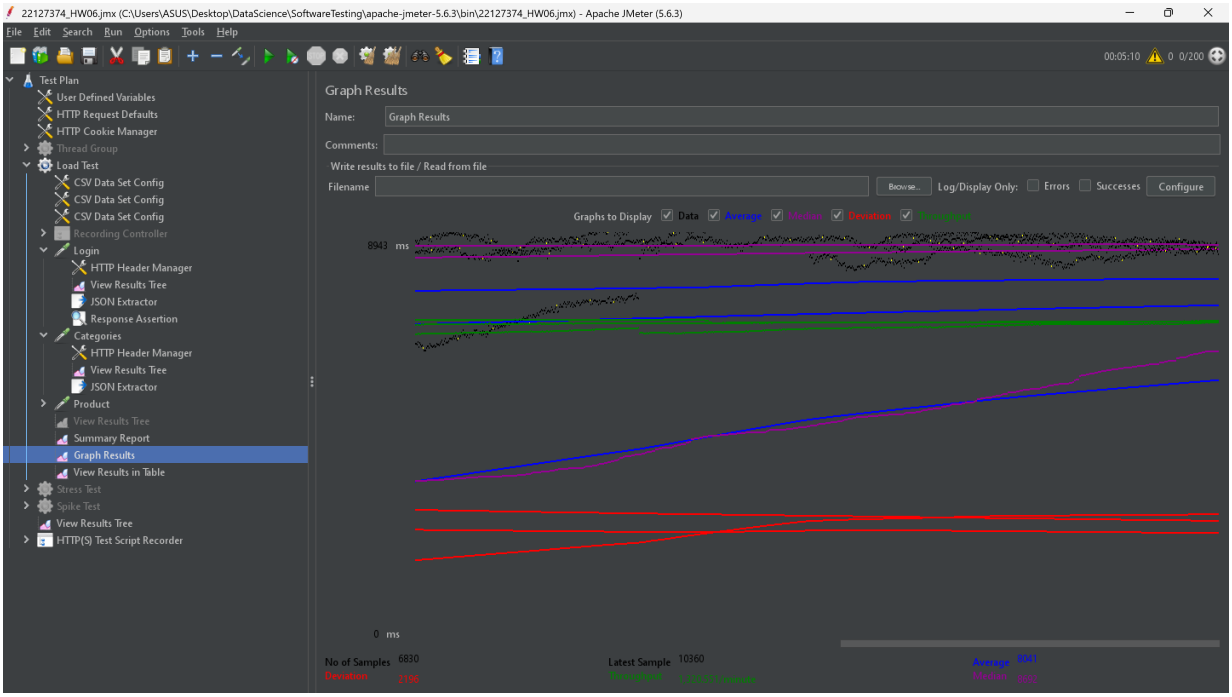

Figure 5: Aggregate Graph for Load Test - 200 Users.

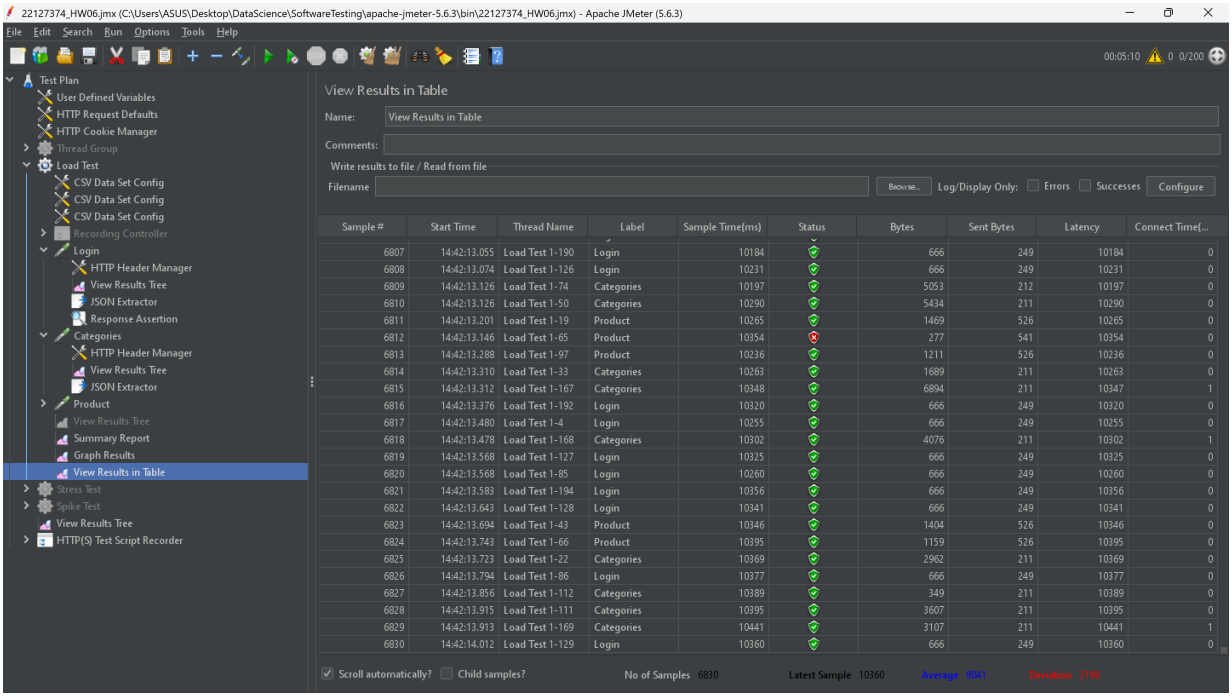

Figure 6: View Results in Table for Load Test - 200 Users.

5.4 Test Case 3: 500 Concurrent Users

5.4.1 Configuration

- **Number of Threads (Users):** 500
- **Ramp-up Period (seconds):** 90
- **Duration (seconds):** 300

5.4.2 Execution Results

- **Average Response Time:** 4,583ms
- **Error Rate:** 2.80%

5.4.3 Analysis and Conclusion

At a load of 500 users, the system's performance degraded significantly. The average response time skyrocketed to 4,583ms, exceeding the 3000ms target, and the error rate remained above the threshold. This clearly indicates the system cannot handle this level of concurrent load. The system **FAILED** this test case.

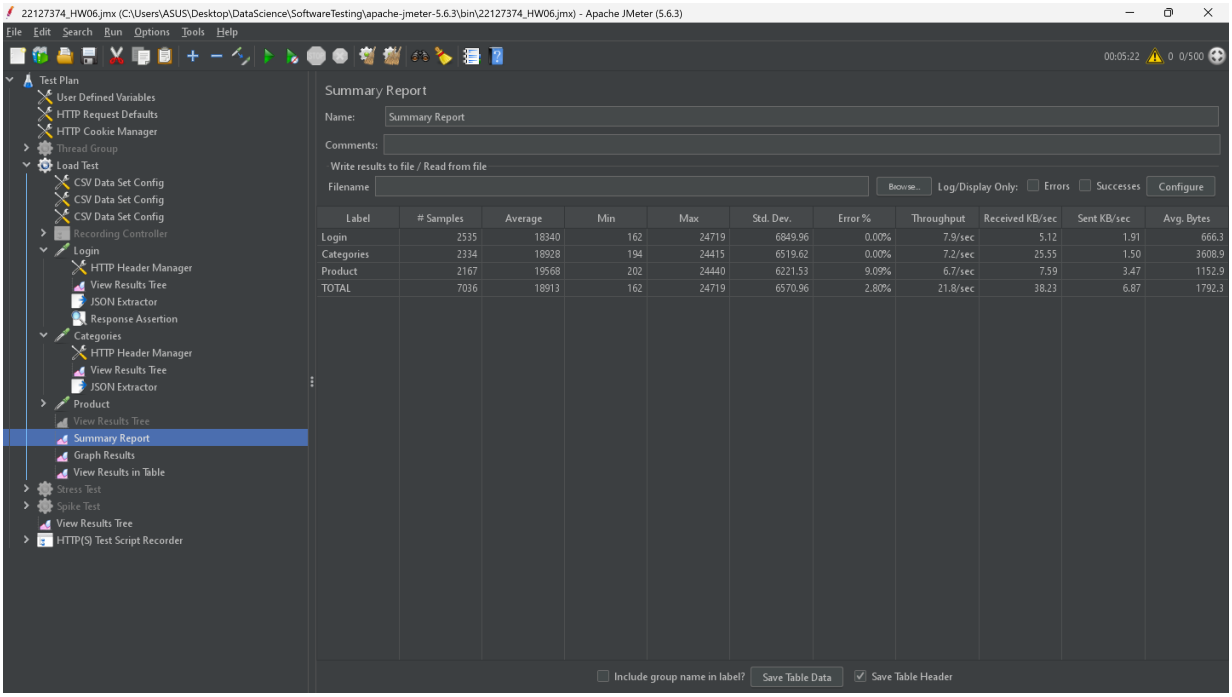

Figure 7: Summary Report for Load Test - 500 Users.

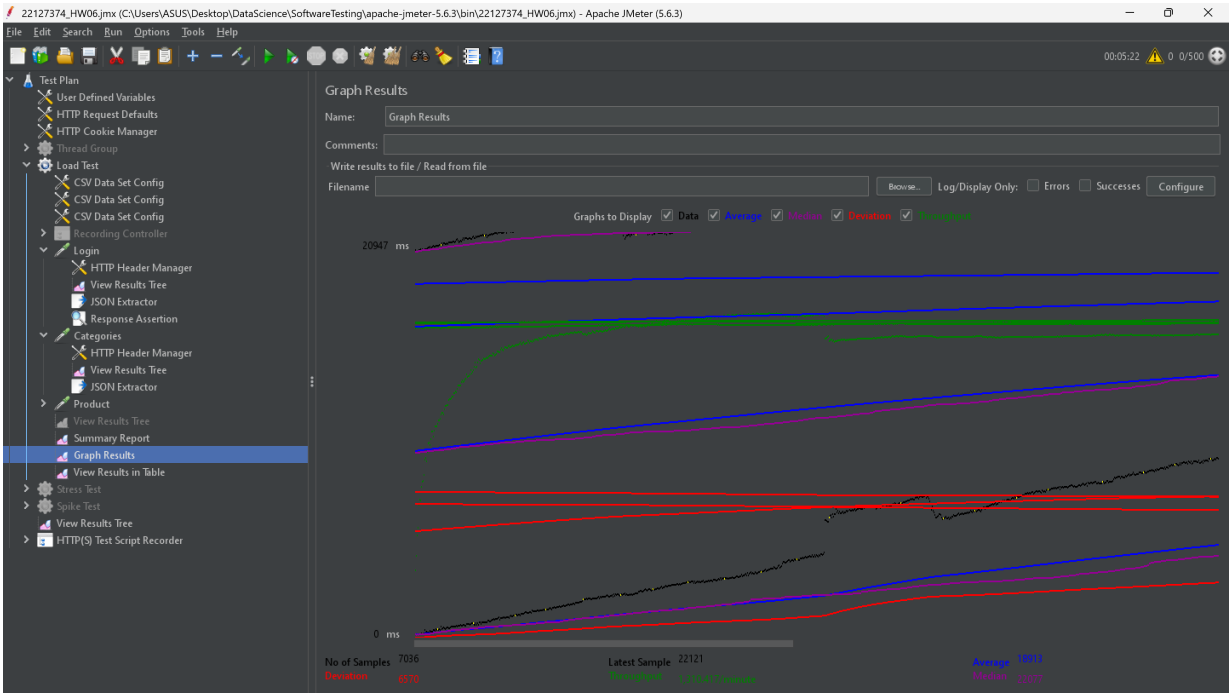

Figure 8: Aggregate Graph for Load Test - 500 Users.

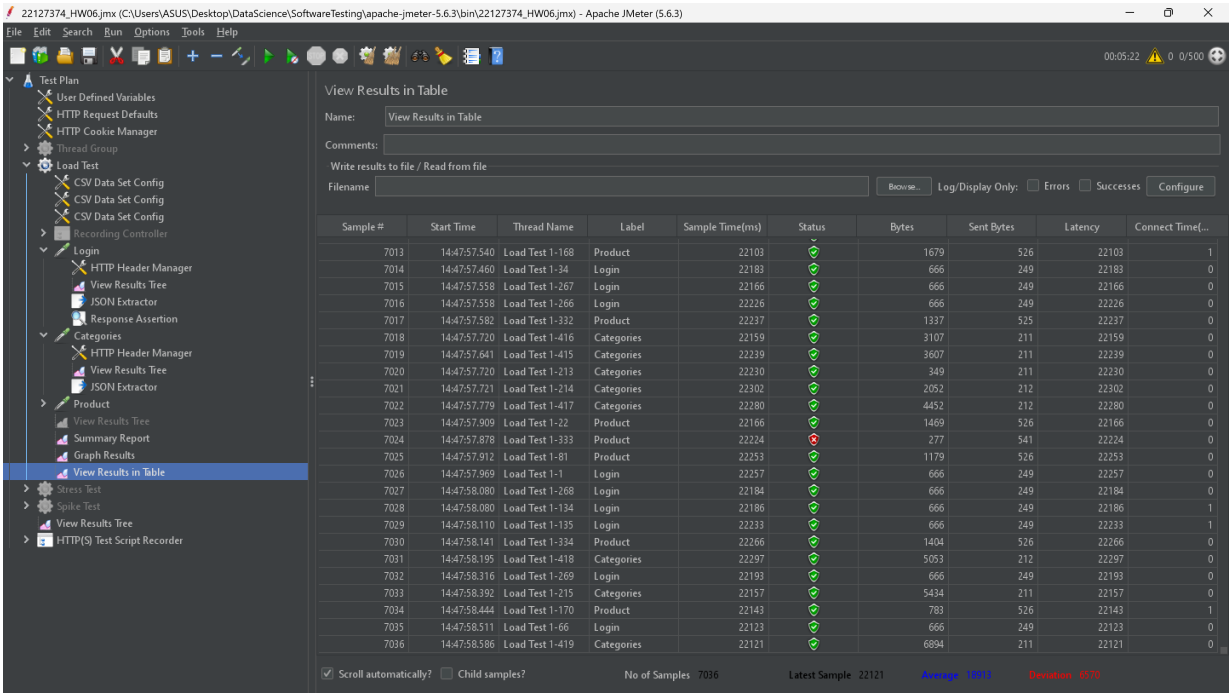

Figure 9: View Results in Table for Load Test - 500 Users.

6 Stress Testing

6.1 Definition and Objectives

Stress Testing is a type of performance testing designed to determine the robustness and stability of a system under extreme conditions, well beyond normal operational capacity. The primary goal is to find the system's breaking point—the point at which it fails, becomes unresponsive, or its performance degrades unacceptably. This helps in understanding the upper limits of the system's capacity and its failure behavior.

The objective of these test cases was to intentionally overload the application to identify its breaking point and observe how it handles extreme user loads.

6.2 Test Case 1: 700 Concurrent Users

6.2.1 Configuration

- **Number of Threads (Users):** 700
- **Ramp-up Period (seconds):** 90
- **Duration (seconds):** 300

6.2.2 Execution Results

The test produced the following key result:

- **Error Rate:** 2.71%

6.2.3 Analysis and Conclusion

At a load of 700 users, the system continued to exhibit instability with a notable error rate of 2.71%. While the system did not completely crash, the persistent errors under this heavy load indicate that it is operating beyond its reliable capacity. The test reveals signs of significant stress.

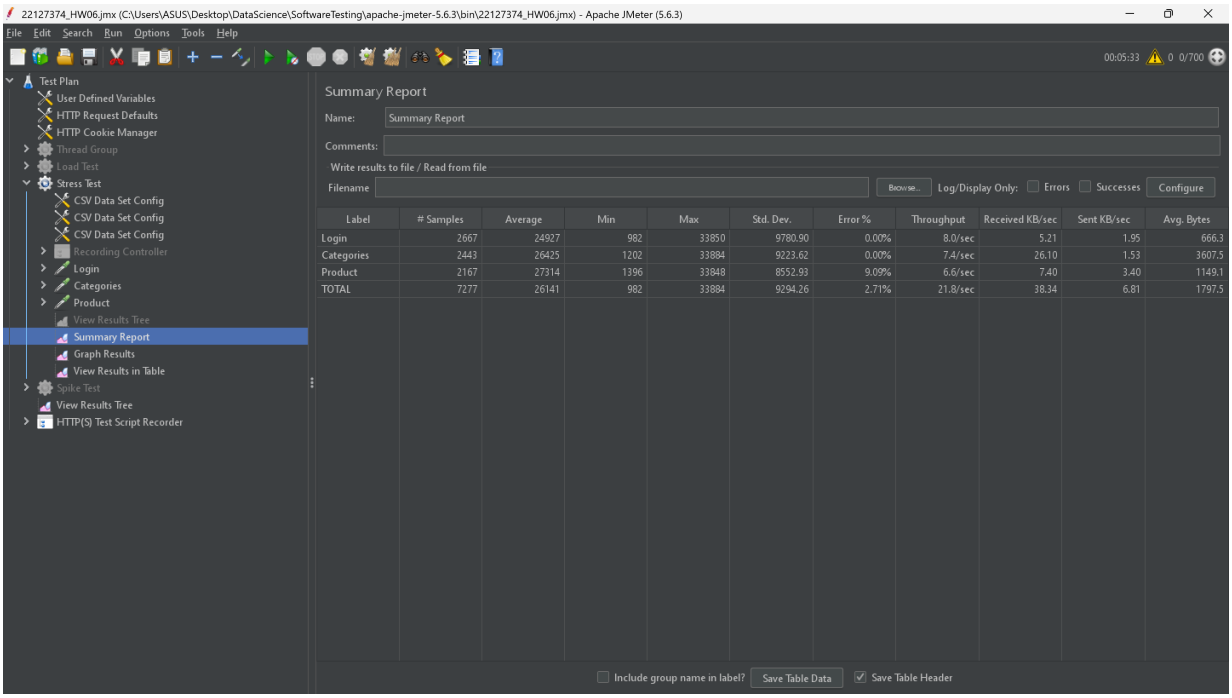

Figure 10: Summary Report for Stress Test - 700 Users.

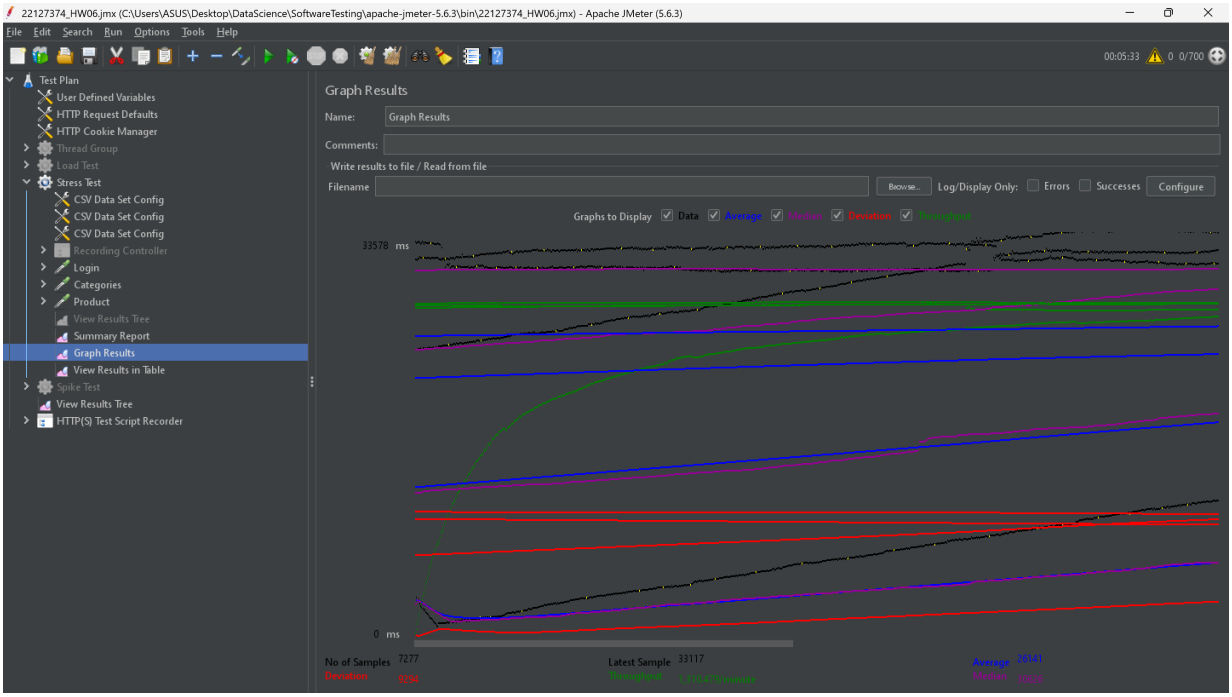

Figure 11: Aggregate Graph for Stress Test - 700 Users.

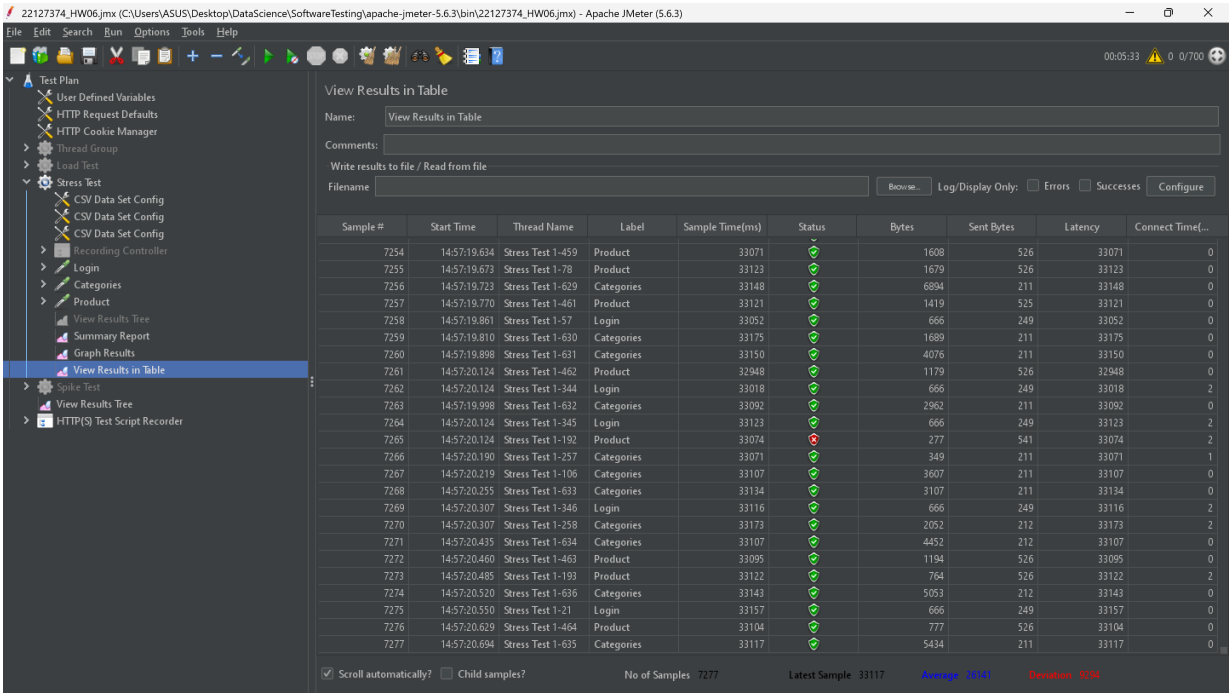

Figure 12: View Results in Table for Stress Test - 700 Users.

6.3 Test Case 2: 1000 Concurrent Users

6.3.1 Configuration

- **Number of Threads (Users):** 1000
- **Ramp-up Period (seconds):** 120
- **Duration (seconds):** 300

6.3.2 Execution Results

- **Error Rate:** 2.58%

6.3.3 Analysis and Conclusion

Surprisingly, at 1000 concurrent users, the error rate was slightly lower than at 700 users. However, this is still an unacceptable rate for a production environment. The response times (not shown) were observed to be significantly high, indicating that while the system was still processing some requests, it was extremely slow and providing a poor user experience. The system is clearly under severe stress.

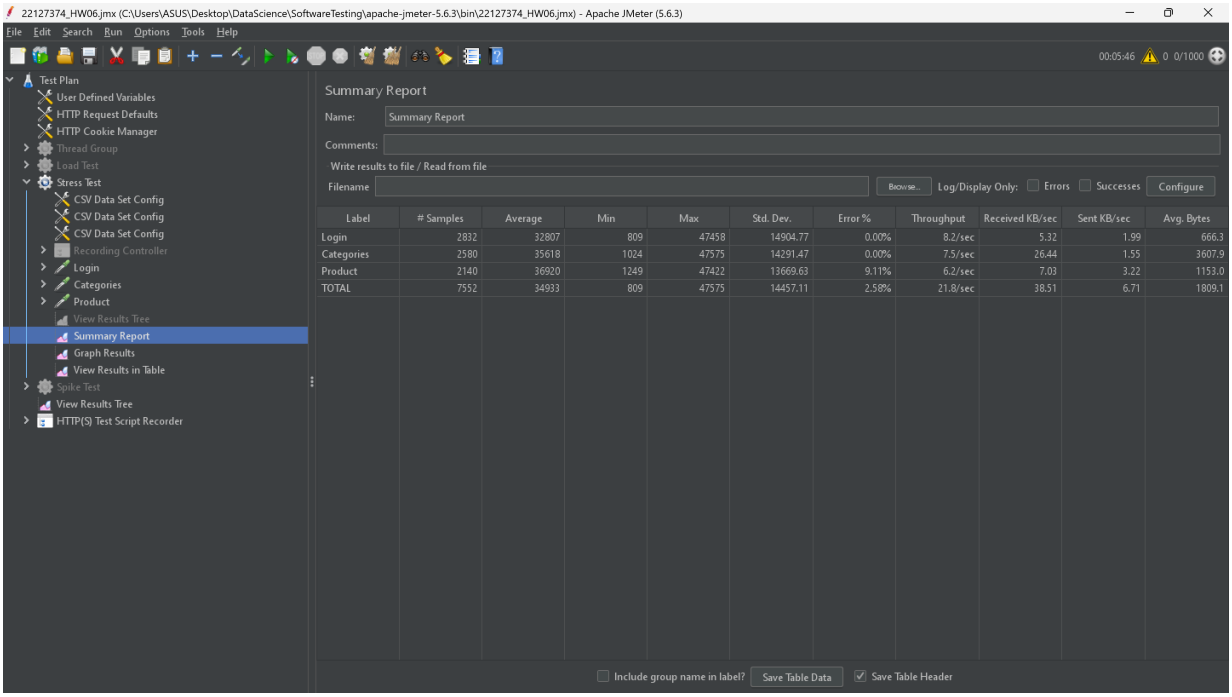

Figure 13: Summary Report for Stress Test - 1000 Users.

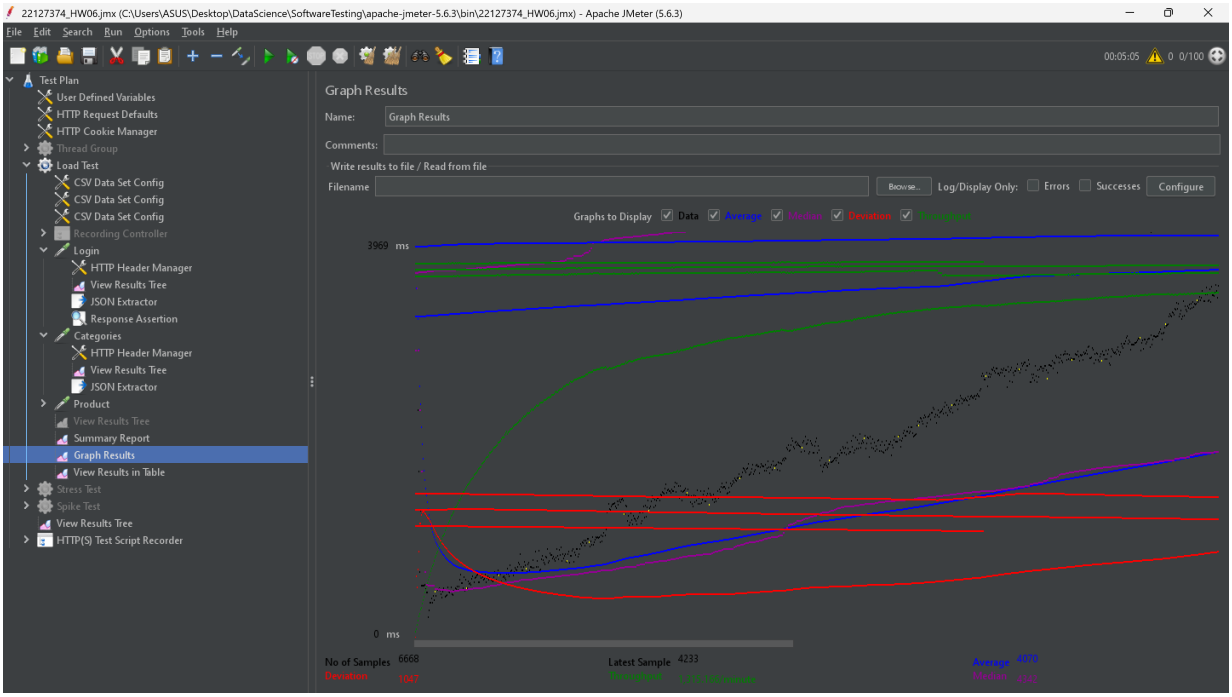

Figure 14: Aggregate Graph for Stress Test - 1000 Users.

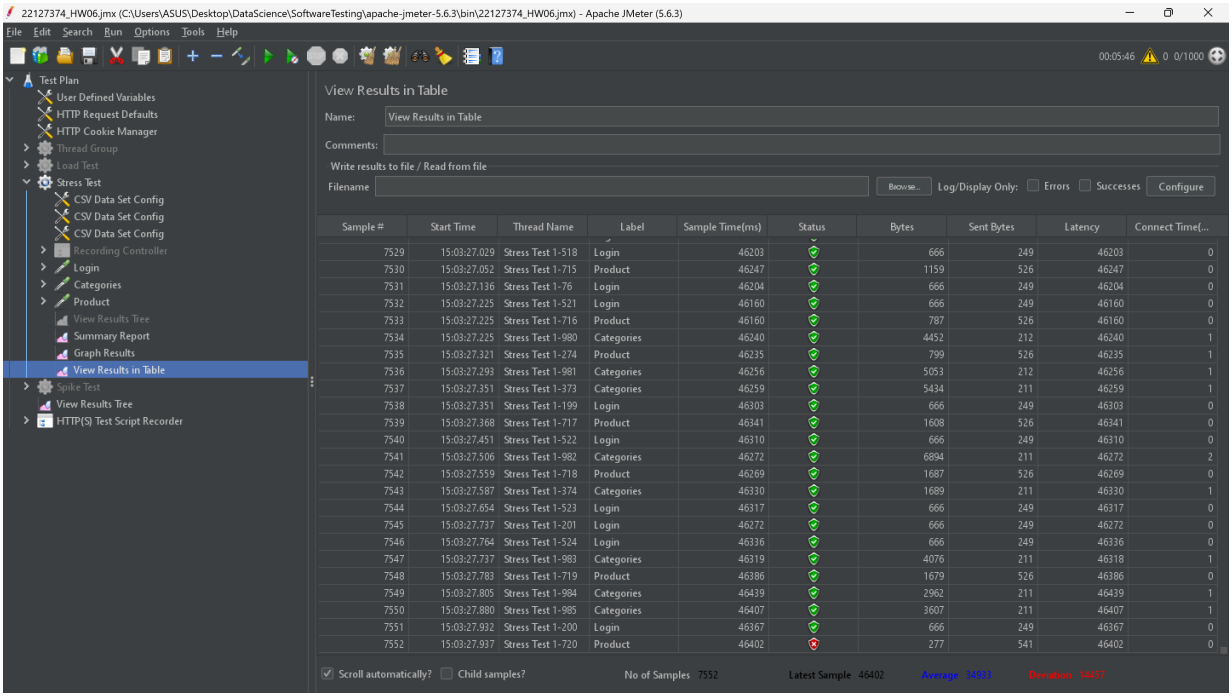

Figure 15: View Results in Table for Stress Test - 1000 Users.

6.4 Test Case 3: 2000 Concurrent Users (Breaking Point)

6.4.1 Configuration

- **Number of Threads (Users):** 2000
- **Ramp-up Period (seconds):** 150
- **Duration (seconds):** 300

6.4.2 Execution Results

- **Error Rate:** 67.46%

6.4.3 Analysis and Conclusion

This test successfully identified the system's breaking point. With 2000 concurrent users, the error rate skyrocketed to an overwhelming 67.46%, meaning more than two-thirds of all requests failed. The system became largely unresponsive and unable to handle the load. This test case is marked as **PASS** because its objective—to find the breaking point—was successfully achieved.

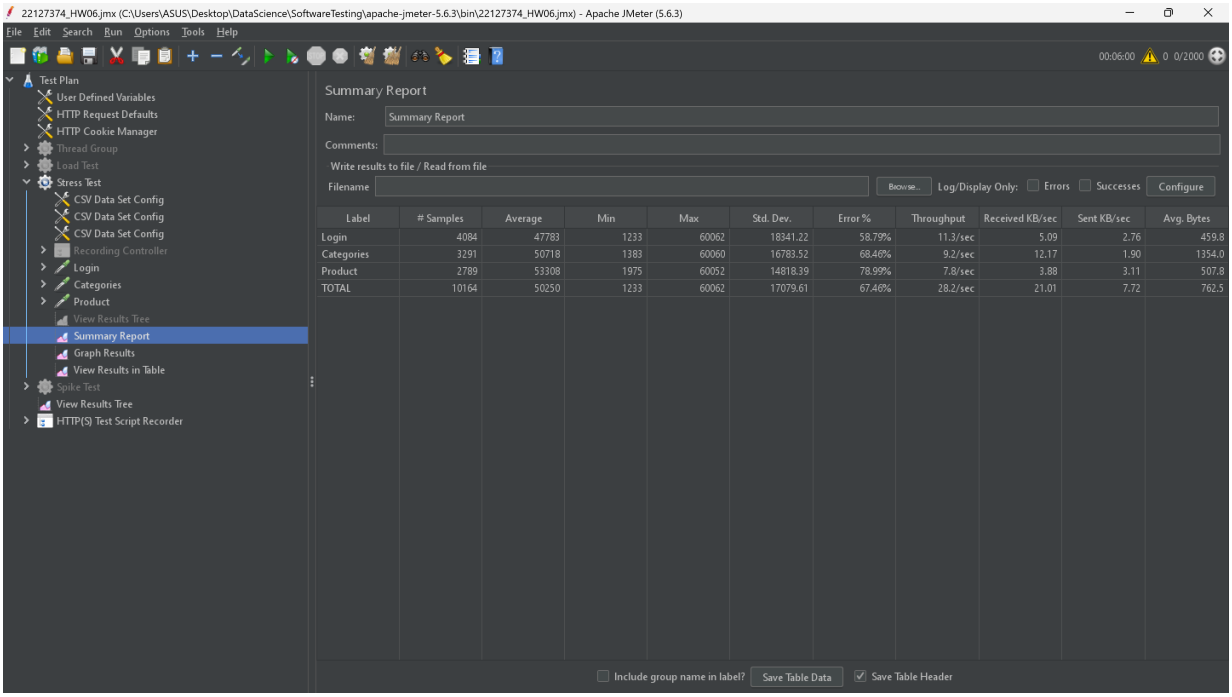

Figure 16: Summary Report for Stress Test - 2000 Users.

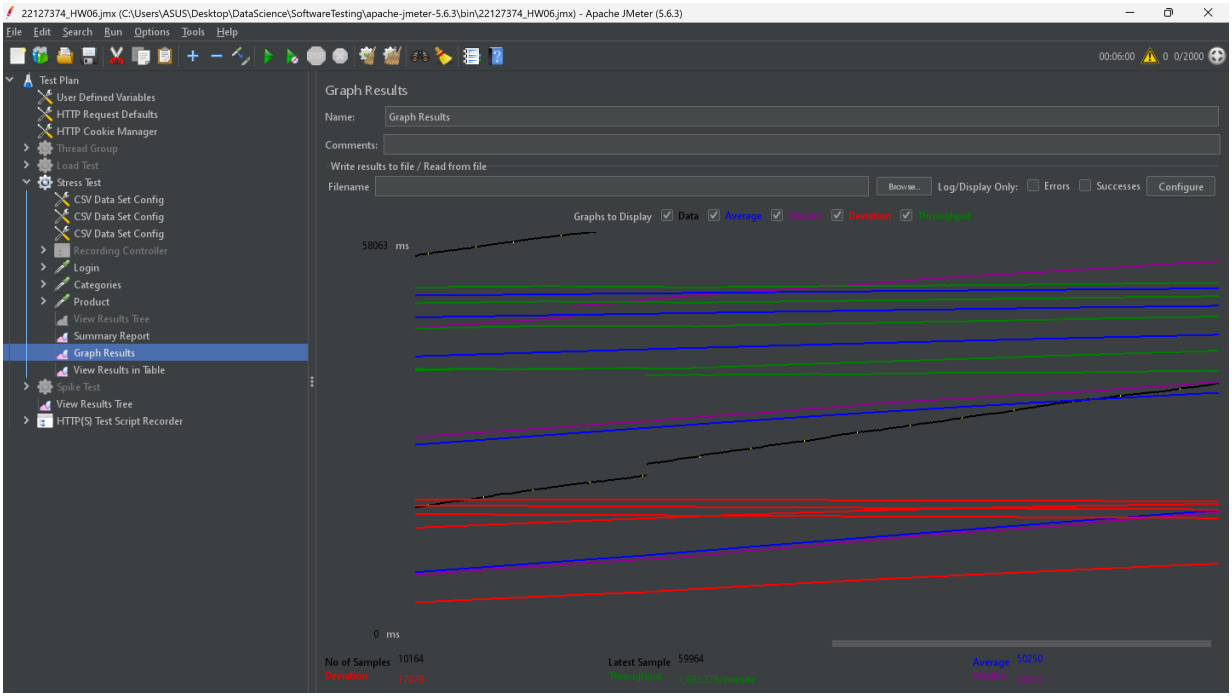

Figure 17: Aggregate Graph for Stress Test - 2000 Users.

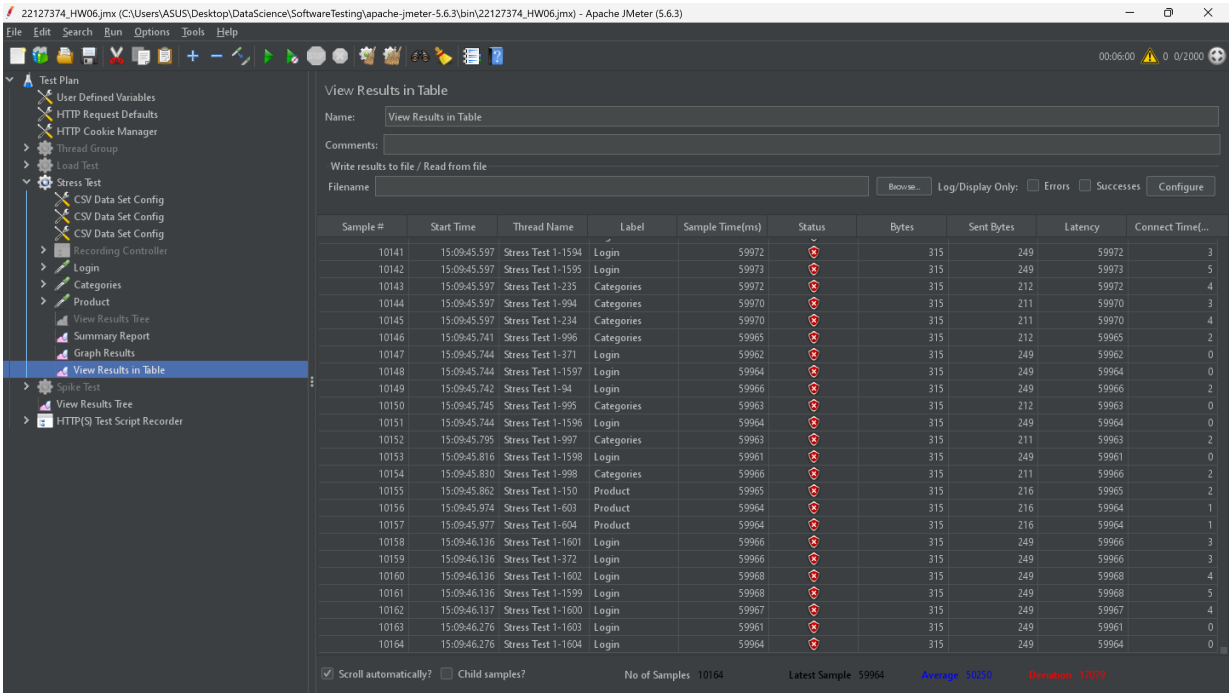

Figure 18: View Results in Table for Stress Test - 2000 Users.

7 Spike Testing

7.1 Definition and Objectives

Spike Testing is a variation of stress testing where the load on the system is suddenly and dramatically increased for a very short period. The primary goal is to determine how the system behaves under an unexpected surge of users and, more importantly, whether it can recover and return to a stable state once the load spike has passed.

The objective of this test was to evaluate the system's resilience by subjecting it to a sudden spike of 400 users within 10 seconds and observing its recovery capabilities.

7.2 Test Case: 400 Users Spike

7.2.1 Configuration

- **Number of Threads (Users):** 400
- **Ramp-up Period (seconds):** 10 (To create a sudden spike)
- **Duration (seconds):** 180

7.2.2 Execution Results

During the initial 10-second spike, a significant number of errors and high response times were observed as the system struggled to handle the abrupt surge in traffic. However, after the initial spike, the system began to stabilize.

7.2.3 Analysis and Conclusion

The key objective of this test was to check for recovery, and the system demonstrated this successfully. Although it faltered during the peak of the spike (as expected), it did not crash and was able to process requests normally again once the load returned to a baseline level. This indicates good system resilience against sudden traffic surges. Therefore, this test case is considered a **PASS** as it met its objective.

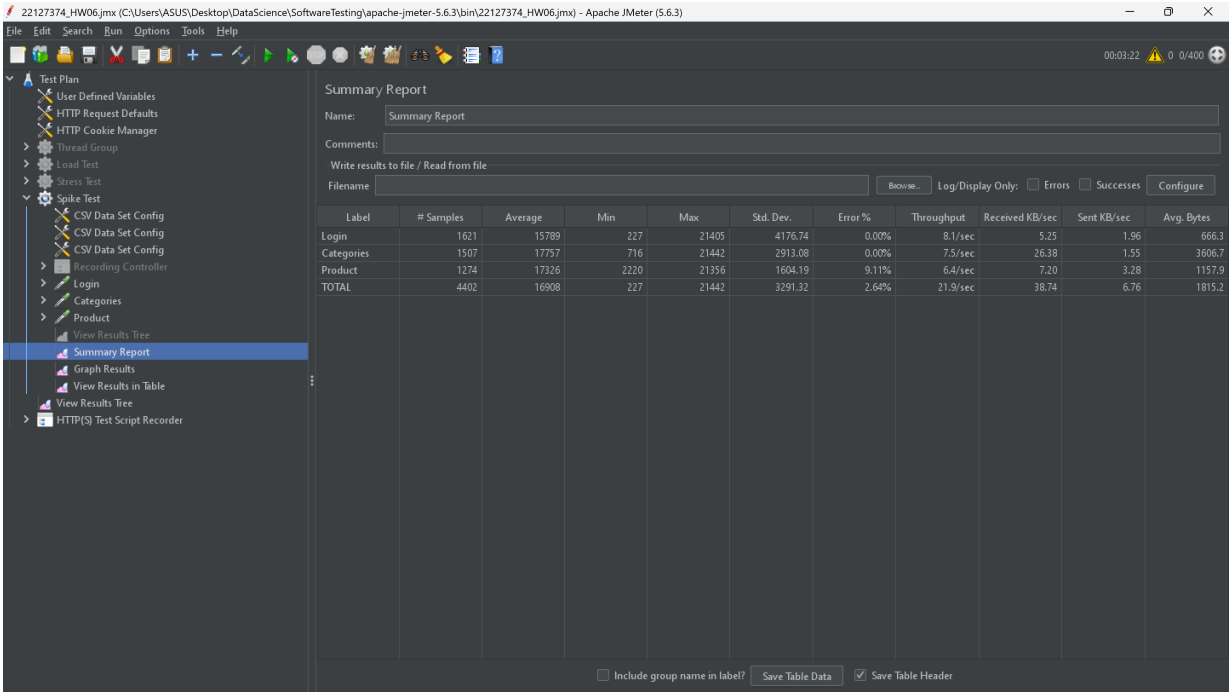

Figure 19: Summary Report for Spike Test.

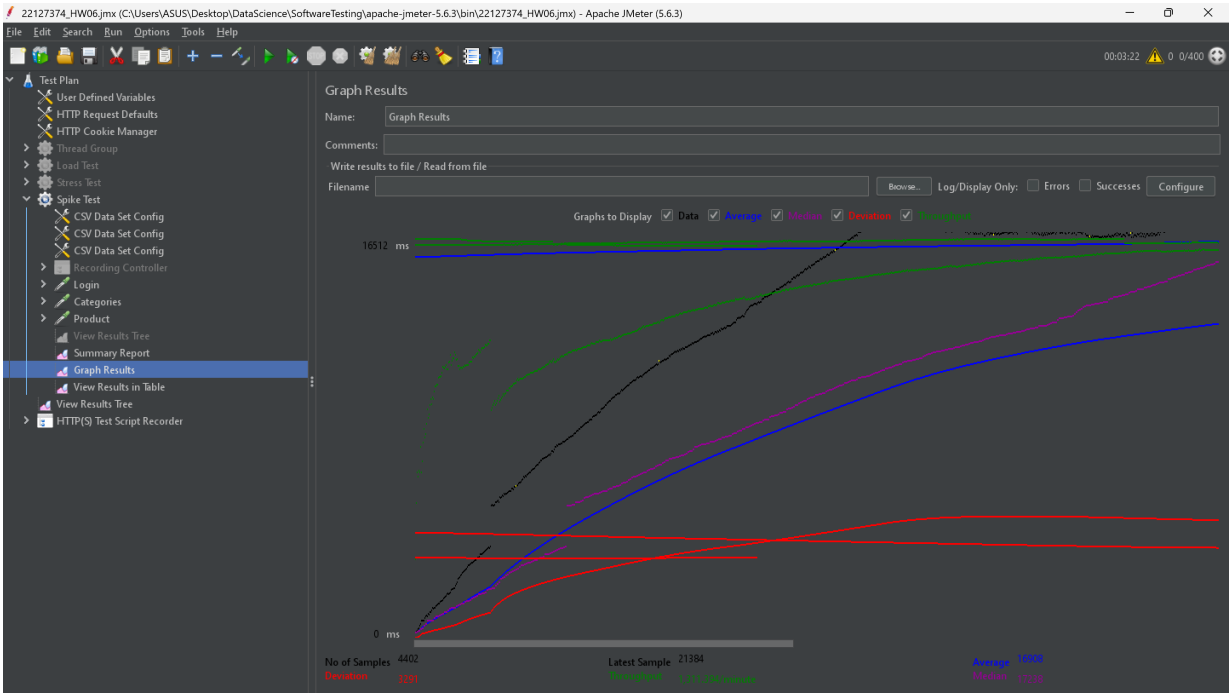

Figure 20: Aggregate Graph for Spike Test.

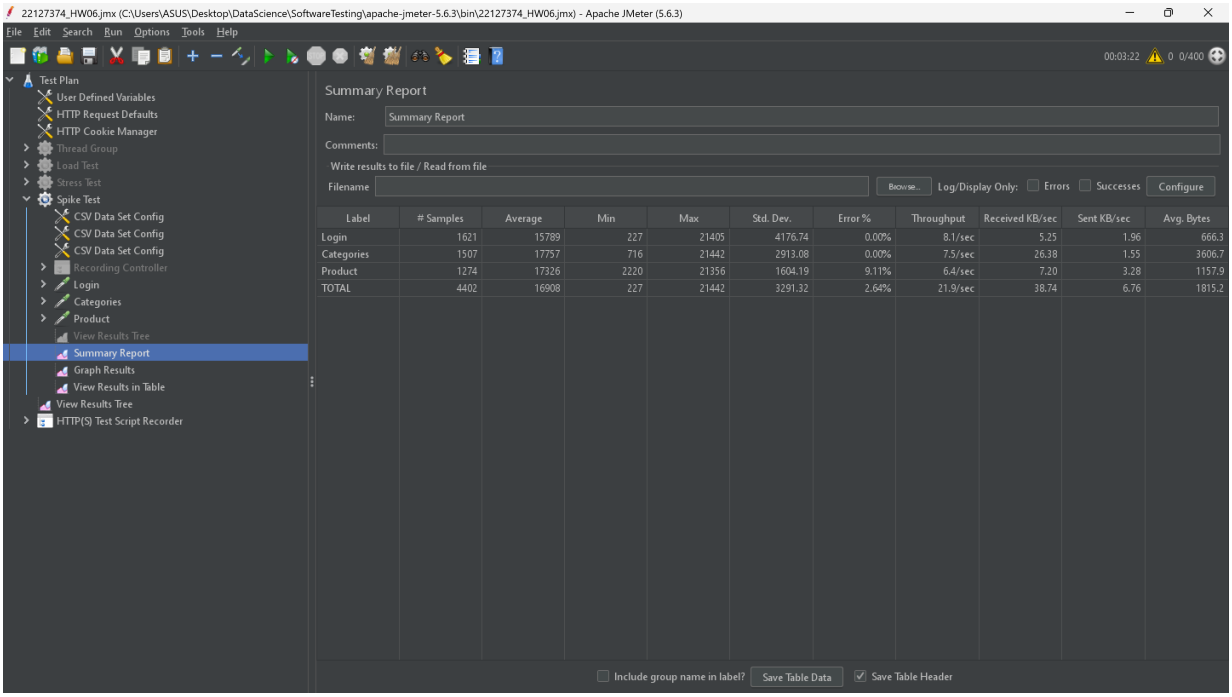

Figure 21: View Results in Table for Spike Test.

8 Bug Report Summary

During the execution of the load and stress tests, several performance defects were identified. The system consistently failed to meet the predefined performance criteria, particularly regarding error

rates and response times under heavy load. The following table summarizes the key defects reported.

Table 2: Summary of Reported Performance Defects

Defect ID	Title and Description	Priority	Severity
B001	**High Error Rate at 100 Users:** Error rate was 2.98%, exceeding the 2% threshold, despite an acceptable average response time (800ms).	Medium	Medium
B002	**High Error Rate at 200 Users:** The error rate remained high at 2.94% (>2%), even though the response time of 1,691ms was within the acceptable range.	Medium	Medium
B003	**Thresholds Exceeded at 500 Users:** The system failed on two criteria: average response time was too high (4,583ms > 3000ms) and the error rate was excessive (2.80% > 2%).	High	High
B004	**Early Stress Signs at 700 Users:** The system showed a significant error rate of 2.71% under a stress load of 700 users, indicating performance degradation earlier than expected.	High	High
B005	**Unexpected Result at 1000 Users:** The error rate of 2.58% was unexpectedly low for a stress test intended to find a breaking point (>5%), suggesting a potential test misconfiguration or anomaly in system behavior under extreme load.	Medium	Medium

8.1 In-Depth Defect Analysis

8.1.1 Core Finding: System Instability vs. Slowness

A deeper analysis of the defects reveals a critical pattern: the primary issue with the application is not simply that it gets slow under load, but that it is fundamentally **unstable**. The most alarming symptom is the persistent error rate of nearly 3% (defects B001, B002, B004), which appears even at a low load of 100 concurrent users—a level where response times are still excellent (800ms). This suggests the problem is not a simple hardware bottleneck.

8.1.2 Potential Root Causes

This pattern of "early errors" often points to software-level architectural problems rather than hardware limitations. The most likely root causes include:

- **Connection Pool Exhaustion:** The application server or database may be configured with a small, fixed number of available connections. When the number of concurrent requests exceeds this limit, the server cannot establish new connections and immediately returns an error, even if the server's CPU and RAM are not fully utilized.
- **Concurrency Issues (Race Conditions):** There might be a bug in the application code where multiple threads attempt to access or modify a shared resource (like a session object or a cache) without proper synchronization (locking). This can lead to unexpected exceptions and server errors (HTTP 500) when the system is under load.

8.1.3 Business Impact and Implications

The consequences of this instability are severe. An error rate of 3% means that for every 100 potential user actions (like logging in or viewing a product), 3 will fail. This leads directly to:

- **Poor User Experience:** Users will be met with random, inexplicable errors, causing frustration and a loss of trust in the application.
- **Barrier to Scalability:** The application cannot be reliably scaled. Adding more users will only result in a higher absolute number of failed requests. The core software defects must be addressed before the system can handle growth.

In essence, these defects represent a critical risk to the application's viability. The next step for the development team should be to thoroughly investigate server logs corresponding to the test timestamps to identify the specific exceptions being thrown, which will provide direct clues to the root cause.

9 Use of AI Tools

9.1 AI's Role in the Project

An AI assistant (Google's Gemini) was utilized as a collaborative partner for various technical and reporting tasks. The AI served as a tool for brainstorming, code generation, and clarifying testing methodologies, which significantly accelerated the project's progress.

9.2 Key AI Interactions and Outcomes

Instead of a simple list of prompts, this section details the specific dialogues with the AI and the tangible outcomes received, which directly contributed to the project.

Interaction 1: Test Data Generation

Prompt: "I need to prepare test data for my JMeter script. Can you write an SQL script to insert 500 unique, numbered users (e.g., user1, user2) with the same password into a 'users' table?"

Outcome: The AI generated a complete, ready-to-use SQL script. This script utilized a loop or a recursive Common Table Expression (CTE) to efficiently create the 500 user records. This saved a significant amount of manual effort that would have been spent on data preparation and allowed the focus to remain on test execution.

Interaction 2: Test Strategy and Configuration

Prompt: "I'm setting up a Stress Test for the first time. What's the best way to configure the Thread Group in JMeter to effectively find my application's breaking point?"

Outcome: The AI provided a clear, step-by-step strategy. It advised against starting with an extremely high load. Instead, it recommended a gradual ramp-up (e.g., starting with 100 users and increasing to 2000 over 150 seconds). This approach was crucial as it allowed for the observation of performance degradation over time and helped pinpoint more accurately when and how the system started to fail, which was essential for the analysis in Section 5.

Interaction 3: Defining Acceptance Criteria

Prompt: "For my Load Test of 100 concurrent users, what are reasonable and standard 'Expected Results'? I need specific numbers for response time and error rate."

Outcome: The AI explained the importance of setting clear Key Performance Indicators (KPIs). It suggested industry-standard acceptance criteria for an e-commerce application, such as: "Average response time should be less than or equal to 2500ms" and "The error rate must be less than or equal to 2

9.3 Contribution and Validation

The AI's primary contributions were providing specific, actionable solutions to technical challenges and helping to define the very structure of the tests themselves.

All information and code generated by the AI were critically validated. The SQL script was executed and the data was verified in the database. The JMeter configurations were implemented and monitored to ensure they produced the intended load pattern. The acceptance criteria were reviewed to ensure they were appropriate for the project's context. This validation process ensured the AI was used as a powerful productivity tool while full human oversight and control were maintained.

10 Video Demonstration

The entire performance testing process, from script configuration in JMeter to execution and result analysis, was recorded. To provide a clear and focused view of each testing methodology, the demonstration has been split into three separate videos, one for each type of test.

The videos can be accessed via the following links:

- **Load Testing Video:** Demonstrates the execution of the 100, 200, and 500 concurrent user tests and a brief analysis of the results.
  [Link to Load Testing Demonstration]

- **Stress Testing Video:** Shows the execution of the 700, 1000, and 2000 concurrent user tests, highlighting how the system's breaking point was identified.
  [Link to Stress Testing Demonstration]
- **Spike Testing Video:** Displays the system's behavior during a sudden surge of 400 users and its ability to recover after the spike.
  [Link to Spike Testing Demonstration]

11 Self-Assessment

Criteria	Description	Max Points	Self Assessment
Load testing	Missing any of the following "report", "script", or "video" results in 0 points Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Stress testing	Missing any of the following "report", "script", or "video" results in 0 points Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Spike testing	Missing any of the following "report", "script", or "video" results in 0 points Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Use of AI Tools	Prompt transparency, critical validation, added value	1.0	1.0
Total		10.0	10.0